Programming Fundamentals

Week 6B – Building a CRUD Application

Programming Fundamentals Team

SETU

2026-10-08

Last time...

We combined lists, dictionaries, loops, selection and functions.

We also met CRUD: **Create, Read, Update, Delete**.

Today we build a small application using those ideas.

Our starting data

```
dogs = [
    {"id": 1, "name": "Buddy", "breed": "Retriever", "age": 4},
    {"id": 2, "name": "Luna", "breed": "Labrador", "age": 2}
]
```

Why is an ID more reliable than searching only by name?

Design the operations

Before writing code, decide:

- Create: what information must we ask for?
- Read: all dogs, or one dog?
- Update: how do we identify the correct dog?
- Delete: how do we avoid removing the wrong one?

Pair discussion: What errors should we anticipate?

One helper function

```
def find_dog(dogs, dog_id):
    for dog in dogs:
        if dog["id"] == dog_id:
            return dog
    return None
```

Predict: What does `find_dog(dogs, 99)` return?

READ: Display all dogs

```
def display_dogs(dogs):
    for dog in dogs:
        print(dog["id"], dog["name"], dog["age"])
```

Challenge: What message would be helpful if there are no dogs?

CREATE: Add a dog

```
def add_dog(dogs):
    dog_id = int(input("ID: "))
    if find_dog(dogs, dog_id) is not None:
        print("That ID already exists")
        return
    name = input("Name: ")
    breed = input("Breed: ")
    age = int(input("Age: "))
    dogs.append({"id": dog_id, "name": name,
                 "breed": breed, "age": age})
```

Why do we check the ID **before** adding?

UPDATE: Change an age

```
def update_dog(dogs):
    dog_id = int(input("ID to update: "))
    dog = find_dog(dogs, dog_id)
    if dog is None:
        print("Dog not found")
    else:
        dog["age"] = int(input("New age: "))
```

Why does changing `dog` also change the record in `dogs`?

DELETE: Remove a dog

```
def delete_dog(dogs):
    dog_id = int(input("ID to delete: "))
    dog = find_dog(dogs, dog_id)
    if dog is None:
        print("Dog not found")
    else:
        dogs.remove(dog)
```

Discuss: How could we add a confirmation step?

A menu brings it together

```
choice = ""
while choice != "5":
    print("1. View dogs")
    print("2. Add dog")
    print("3. Update dog")
    print("4. Delete dog")
    print("5. Exit")
    choice = input("Choose: ")
    # Call the appropriate function here
```

Which topics from previous weeks are we reusing?

Predict, test, improve

Test these situations:

1. View all dogs.
2. Add a new dog.
3. Add a duplicate ID.
4. Update an unknown ID.
5. Delete a dog, then search for it.

Question: What happens if the user types `hello` when asked for an age?

From dogs to questions

Our Dog Day Care program manages dog records.

Our expert system will manage **question records** and **rule records**.

The structure of the solution is surprisingly similar.

Expert system: READ

```
def display_questions(questions):
    for question in questions:
        print(question["id"], question["text"])
```

Predict: What changes compared with `display_dogs()`?

What stays exactly the same?

Expert system: CREATE

```
def add_question(questions, question_id, text):
    for question in questions:
        if question["id"] == question_id:
            return False
    questions.append({"id": question_id, "text": text})
    return True
```

Discuss: Why might returning `True` or `False` be useful?

Expert system: UPDATE and DELETE

Work with a partner and write **pseudocode** for:

- `update_question(questions, question_id, new_text)`
- `delete_question(questions, question_id)`

Both must handle an ID that does not exist.

Hint: Could a `find_question()` helper make this easier?

Knowledge and program logic

Our goal is to keep two things separate:

Knowledge: questions, rules and recommendations.

Program logic: code that displays, searches, edits and eventually evaluates those rules.

Later, adding a new rule should not usually require another `if/elif` in the decision engine.

One problem remains…

We can add and edit questions while the program is running.

But what happens when we close Python and start again?

The changes disappear.

Next week: files and JSON — making our knowledge base persistent.